23. Linked Structures, Stacks, and Queues

23. Linked Structures, Stacks, and Queues

Phase: Phase 4 — Algorithms and data structures
Estimated time: 1–2 days
Slug: `23-linked-stacks-queues`

Learning objectives

- Implement stack and queue with slices or lists
- Know when linked lists help vs hurt
- Reason about amortized append costs
- Use containers/list only when needed

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Linked structures, stacks, and queues appear in parsers, schedulers, and BFS. Implementing them in Go clarifies pointers and ownership.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Nodes hold values and pointers to other nodes.
- Stacks: push/pop at one end; queues: FIFO.
- Nil terminates lists; always check before deref.
- Slices often beat linked lists for cache locality.
- Know both: interviews and specialized allocators still use links.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-linked && cd /tmp/gw-linked
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import "fmt"

type Stack []int

func (s *Stack) Push(v int) { *s = append(*s, v) }
func (s *Stack) Pop() int {
    v := (*s)[len(*s)-1]
    *s = (*s)[:len(*s)-1]
    return v
}

func main() {
    var s Stack
    s.Push(1); s.Push(2)
    fmt.Println(s.Pop(), s.Pop())
}
```

Run it:

```
go run .
```

Expected output:

```
2 1
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Implement stack with a slice backing.
2. Implement a queue with head index or container/list.
3. Draw pointer boxes for a 3-node linked list.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Stack (LIFO)

Slice-backed stacks are idiomatic and cache-friendly:

```
type Stack []int
func (s *Stack) Push(v int) { *s = append(*s, v) }
func (s *Stack) Pop() (int, bool) {
    if len(*s) == 0 { return 0, false }
    v := (*s)[len(*s)-1]
    *s = (*s)[:len(*s)-1]
    return v, true
}
```

Queue (FIFO)

Can use slice with head index, ring buffer, or two stacks. Avoid O(n) `dequeue` from front without strategy.

Linked lists

Useful for O(1) splice given a node; often slower than slices due to pointer chasing. Prefer slices first.

Failure modes to watch while coding

- Always reaching for linked lists from textbook habit
- Queue via slice[1:] in hot path causing GC pressure
- Forgetting empty checks on Pop

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- collections.deque ≈ efficient queue; Go slices work if careful.
- No pointers in Python — references everywhere.
- Linked lists are rarer in both languages for general storage.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

Prefer slice-based stacks/queues unless you need mid-list splicing.

Practice checklist

Before the exercise, confirm you can:

1. Explain "Linked Structures, Stacks, and Queues" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Always reaching for linked lists from textbook habit
- Queue via slice[1:] in hot path causing GC pressure
- Forgetting empty checks on Pop

Exercises

Work in `exercises/23-linked-stacks-queues/`. Read that folder's `README.md` before you code.

What you will build

Implement stack operations required by tests (push/pop/peek).

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/23-linked-stacks-queues/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. LIFO vs FIFO?
2. Why are slices often faster than lists?
3. Amortized cost of append?

Further reading

- https://pkg.go.dev/container/list